

Day 18 – Shell Scripting: Functions, Strict Mode & Reusable Scripts

Objective

Learn how to:

- Create and use functions
 - Pass arguments to functions
 - Use local variables
 - Enable strict mode with `set -euo pipefail`
 - Build reusable and maintainable shell scripts
-

Task 1: Basic Functions

What is a Function?

A function is a reusable block of code that performs a specific task.

functions.sh

Script

```
None
#!/bin/bash

greet() {
    echo "Hello, $1!"
}

add() {
    echo "Sum: $(( $1 + $2 ))"
}
```

```
greet "Parthavi"  
add 10 20
```

Output

```
None  
Hello, Parthavi!  
Sum: 30
```

```
root@ip-172-31-44-9:/home/ubuntu# chmod +x function.sh  
root@ip-172-31-44-9:/home/ubuntu# ./function.sh  
Hello, Parthavi!  
Sum: 30  
root@ip-172-31-44-9:/home/ubuntu#
```

Task 2: Functions with Return Values

Why Use Functions?

Functions help organize code into smaller, reusable sections.

disk_check.sh

Script

```
None  
#!/bin/bash  
  
check_disk() {  
    echo "Disk Usage:"  
    df -h /  
}  
  
check_memory() {
```

```

    echo
    echo "Memory Usage:"
    free -h
}

check_disk
check_memory

```

Output

```

None
Disk Usage:

Filesystem      Size Used Avail Use% Mounted on
/dev/xvda1      20G  5G   15G  25% /

```

Memory Usage:

```

          total        used        free
Mem:      965Mi      220Mi      500Mi

```

```

root@ip-172-31-44-9:/home/ubuntu# ./disk_check.sh
Disk Usage:
Filesystem      Size  Used Avail Use% Mounted on
/dev/root       6.7G  2.4G  4.3G  36% /

Memory Usage:
          total        used        free      shared  buff/cache   available
Mem:      908Mi      316Mi      169Mi       2.7Mi       537Mi       592Mi
Swap:      0B          0B          0B
root@ip-172-31-44-9:/home/ubuntu#

```

Task 3: Strict Mode

What is Strict Mode?

Strict mode makes scripts safer by stopping execution when errors occur.

strict_demo.sh

Script

```
None
#!/bin/bash

set -euo pipefail

echo "$UNDEFINED_VAR"
```

Output

```
None
UNDEFINED_VAR: unbound variable
```

```
root@ip-172-31-44-9:/home/ubuntu# ./script_demo.sh
./script_demo.sh: line 5: UNDEFINED_VAR: unbound variable
root@ip-172-31-44-9:/home/ubuntu#
```

Example of set -e

Script

```
None
#!/bin/bash

set -e

ls /tmp
ls /nonexistent

echo "This line will not run"
```

Output

None

```
ls: cannot access '/nonexistent': No such file or directory
```

Example of pipefail

Script

None

```
#!/bin/bash

set -o pipefail

cat missingfile.txt | grep test
```

Output

None

```
cat: missingfile.txt: No such file or directory
```

What Does Each Flag Do?

set -e

Exit immediately if any command fails.

set -u

Treat undefined variables as errors.

set -o pipefail

Fail the entire pipeline if any command in the pipeline fails.

Task 4: Local Variables

What is a Local Variable?

A local variable exists only inside a function.

local_demo.sh

Script

```
None
#!/bin/bash

global_var="Global"

local_test() {
    local local_var="Local"
    echo "Inside Function: $local_var"
}

local_test

echo "Outside Function: $global_var"
```

Output

```
None
Inside Function: Local
Outside Function: Global
```

```
root@ip-172-31-44-9:/home/ubuntu# ./local_demo.sh
Inside Function: Local
Outside Function: Global
root@ip-172-31-44-9:/home/ubuntu#
```

Local vs Regular Variables

Script

```
None
#!/bin/bash

demo() {
    my_var="Hello"
}

demo

echo "$my_var"
```

Output

```
None
Hello
```

Observation

Regular variables remain available outside the function.

Local variables are accessible only inside the function.

Task 5: Build a Script – System Info Reporter

system_info.sh

Script

```
None
#!/bin/bash
```

```
set -euo pipefail

hostname_info() {
    echo "==== HOSTNAME & OS ===="
    hostname
    uname -a
    echo
}

uptime_info() {
    echo "==== UPTIME ===="
    uptime
    echo
}

disk_info() {
    echo "==== DISK USAGE ===="
    du -sh /* 2>/dev/null | sort -hr | head -5
    echo
}

memory_info() {
    echo "==== MEMORY USAGE ===="
    free -h
    echo
}

cpu_info() {
    echo "==== TOP CPU PROCESSES ===="
    ps aux --sort=-%cpu | head -6
    echo
}

main() {
    hostname_info
    uptime_info
```



```
        disk_info
        memory_info
        cpu_info
    }

    main
```

Output

```
None
===== HOSTNAME & OS =====
ip-172-31-44-9
Linux Ubuntu 24.04

===== UPTIME =====
up 3 days, 5 hours

===== DISK USAGE =====
2.1G /usr
1.8G /var
950M /home
700M /opt
500M /tmp

===== MEMORY USAGE =====
                total          used          free
Mem:           965Mi         230Mi         500Mi

===== TOP CPU PROCESSES =====
USER          PID %CPU COMMAND
root          1200 12.5 nginx
ubuntu        3400  8.2 docker
```

```
root@ip-172-31-44-9:/home/ubuntu# ./system_info.sh
===== HOSTNAME & OS =====
ip-172-31-44-9
Linux ip-172-31-44-9 7.0.0-1004-aws #4-Ubuntu SMP PREEMPT Mon Apr 13 13:14:24 UTC 2026 x86_64 GNU/Linux

===== UPTIME =====
05:13:14 up 33 min,  2 users,  load average: 0.00, 0.02, 0.00

===== DISK USAGE =====
1.8G      /usr
658M      /snap
554M      /var
102M      /boot
6.4M      /etc
root@ip-172-31-44-9:/home/ubuntu#
```

Key Commands Learned

```
None
function_name() { }
local
set -e
set -u
set -o pipefail
df -h
free -h
du -sh
ps aux
hostname
uname
```

Interview Questions

What is a function in shell scripting?

A reusable block of code that performs a specific task.

Why use functions?

To reduce code duplication and improve readability.

What does local do?

Creates a variable that is accessible only inside a function.

What does set -e do?

Stops script execution when a command fails.

What does set -u do?

Throws an error when an undefined variable is used.

What does set -o pipefail do?

Returns failure if any command in a pipeline fails.

Why is strict mode recommended?

It helps catch errors early and makes scripts more reliable.

What is \$? in shell scripting?

It stores the exit status of the last executed command.

- 0 = Success
 - Non-zero = Failure
-

What I Learned

- Functions make scripts modular and reusable.
- Local variables prevent unwanted variable leakage.
- Strict mode improves script reliability.
- System administration tasks can be organized into reusable functions.
- Writing clean scripts makes troubleshooting and maintenance easier.